

SNOWPRO ADVANCED: ADMINISTRATOR (ADA-C02)

MODULE 1: SECURITY, RBAC, AND USER ADMINISTRATION

This reference manual provides a comprehensive, hands-on masterclass covering **Domain 1.0 (Snowflake Security, Role-Based Access Control (RBAC), and User Administration)**, which represents **31% of the total score** on the SnowPro Advanced: Administrator (ADA-C02) exam blueprint. Designed for candidates focused on balancing deep theoretical principles with bulletproof practical skills, this guide explores physical traversal paths, permission-inheritance flows, managed schemas, and real-world automated governance patterns.

By working through this manual, you will learn how to systematically debug connectivity failures, structure access versus functional roles, and run targeted 'Break & Fix' scenarios inside your active trial sheets to lock in exam-readiness.

EXAM STRATEGY TIP

Do not just memorize commands. The ADA-C02 exam presents highly complex, multi-layered scenarios where you must act as a lead database administrator. You will be tested on how privilege boundaries intersect during database cloning, future grants, and multi-database transactions. Hands-on verification is the only path to passing.

1. System-Defined Administrative Roles

Snowflake implements a hybrid model of **Discretionary Access Control (DAC)** and **Role-Based Access Control (RBAC)**. At the apex of this model are five default, system-defined administrative roles. Managing these roles securely is paramount to passing the Domain 1.1 and 1.4 security guidelines.

System Role	Primary Responsibilities & Boundaries
ACCOUNTADMIN	Apex administrator. Holds administrative access to configure account-level parameters, manage organization profiles, audit billing/credits, and forcefully stop any active SQL statement running across any compute warehouse. *Not a database superuser by default* — requires explicit role mappings to modify child user data.
SECURITYADMIN	Security architect. Holds the global MANAGE GRANTS privilege. This role can view, grant, and revoke any privilege on any object in the system. USERADMIN is nested as its direct child in the standard inheritance tree.
USERADMIN	User and role specialist. Owns user accounts and custom roles that it creates. Can create and modify users, but cannot manage grants on objects created by other roles.
SYSADMIN	Compute and storage creator. Holds privileges to spin up virtual warehouses, databases, schemas, and child tables/views. In production frameworks, all custom role hierarchies roll up directly to SYSADMIN.
PUBLIC	Catch-all role. Automatically granted to every active user. Object privileges mapped to PUBLIC are instantly visible across the entire platform, creating a major risk for accidental data exposure.

Securing administrative boundaries requires adhering to three core operational patterns:

- **The Two-User Rule:** Always grant the ACCOUNTADMIN role to at least two distinct user accounts in production. If an administrator is locked out, resetting an isolated ACCOUNTADMIN profile requires a manual verification by Snowflake Support that can take up to two business days. Mapped backup administrators prevent this emergency.
- **Never Set Default to ACCOUNTADMIN:** To prevent administrators from accidentally executing daily ETL scripts or creating general schemas under the root administrative role, designate their default active session role as SYSADMIN.
- **Isolate Custom Roles:** When a custom role is created, it exists in total isolation. By default, not even ACCOUNTADMIN can drop or modify objects created by custom roles unless the hierarchy is structured to roll up to SYSADMIN, which rolls up to ACCOUNTADMIN.

Establishing a Clean Administrative Rollup

Always execute this standard structural pattern when deploying new roles to keep them visible to your administrative hierarchy:

```
-- Create custom role
USE ROLE USERADMIN;
CREATE ROLE data_analyst_func;
```

```
-- Enforce structural rollup to maintain administrative visibility  
USE ROLE SECURITYADMIN;  
GRANT ROLE data_analyst_func TO ROLE sysadmin;
```

2. Access vs. Functional Roles & Container Traversal

To manage enterprise permissions without scaling overhead or 'privilege bloat', Snowflake recommends segregating custom roles into **Object Access Roles** and **Business Functional Roles**. This pattern separates physical object mapping from human user mapping.

- **Access Roles (Object Level):** Granted direct, granular privileges on raw objects (such as USAGE, SELECT, INSERT). Named to align with databases and environments (e.g., db_sales_prod_r).
- **Functional Roles (Business Level):** Mapped directly to corporate departments or job titles (e.g., analyst, developer). These roles are granted one or more Access Roles, but never hold direct table-level privileges. Users are assigned functional roles.

The physical representation of this mapping shows a clean, decoupled flow:

Physical Object	Granular Privilege	Access Role	Functional Role	Assigned User
sales_table	SELECT	db_sales_r	analyst_func	user_alice
sales_table	INSERT, UPDATE	db_sales_rw	engineer_func	user_bob
billing_table	SELECT	db_billing_r	analyst_func	user_alice

The Container Traversal Rule

A common point of operational failure in access control architecture is forgetting the containment path. To successfully read a table, a role cannot just have table-level permissions. It must be granted **USAGE on the parent Database** and **USAGE on the parent Schema** to traverse down the containers. Without traversal privileges, queries will fail with database-not-found or authorization errors.

EXAM SCENARIO: CLONED OBJECTS & INHERITANCE

Cloning a database or schema creates a brand-new container object in the Snowflake catalog. Any direct privileges granted on the *source container itself* (such as USAGE on the database) do NOT copy over to the cloned container. However, cloned databases and schemas RETAIN all privileges previously granted on the child objects (such as tables and views) nested inside them.

3. Hands-on Lab Exercises (Labs 1 & 2)

The following labs represent the core security structures. Run these scripts sequentially inside a fresh, clean SQL worksheet on your Snowflake trial environment.

Lab 1: The 'Permission Denied' Trap (Container Traversal)

Goal: Prove that direct object SELECT access is useless without traversing parent database and schema container boundaries, and fix the traversal gap.

```
-- 1. Set up objects as SYSADMIN
USE ROLE SYSADMIN;
CREATE DATABASE IF NOT EXISTS hr_prod_db;
CREATE SCHEMA IF NOT EXISTS hr_prod_db.payroll;
CREATE TABLE IF NOT EXISTS hr_prod_db.payroll.salaries (emp STRING, amt INT);
INSERT INTO hr_prod_db.payroll.salaries VALUES ('Alice', 140000), ('Bob', 98000);

-- 2. Build isolated role
USE ROLE SECURITYADMIN;
CREATE ROLE salary_reader;
GRANT SELECT ON TABLE hr_prod_db.payroll.salaries TO ROLE salary_reader;
GRANT ROLE salary_reader TO USER CURRENT_USER();

-- 3. Test query under isolated role
USE ROLE salary_reader;
SELECT * FROM hr_prod_db.payroll.salaries;
--> ERROR: 'Database HR_PROD_DB does not exist or not authorized.'

-- 4. Resolve traversal path
USE ROLE SECURITYADMIN;
GRANT USAGE ON DATABASE hr_prod_db TO ROLE salary_reader;
GRANT USAGE ON SCHEMA hr_prod_db.payroll TO ROLE salary_reader;

-- 5. Test again. Query now succeeds!
USE ROLE salary_reader;
SELECT * FROM hr_prod_db.payroll.salaries;
```

Lab 2: The Rogue Developer (Managed Access Schemas)

Goal: Compare standard schemas (DAC) where object creators hold complete control over grants with Managed Access Schemas where privilege management is strictly centralized.

```
-- 1. Prepare dev roles
USE ROLE SECURITYADMIN;
```

```
CREATE ROLE IF NOT EXISTS dev_role;
GRANT ROLE dev_role TO USER CURRENT_USER();
GRANT ROLE dev_role TO ROLE sysadmin;

-- 2. Standard Schema: Developer holds grant control
USE ROLE SYSADMIN;
CREATE SCHEMA hr_prod_db.standard_sandbox;
GRANT USAGE ON DATABASE hr_prod_db TO ROLE dev_role;
GRANT USAGE, CREATE TABLE ON SCHEMA hr_prod_db.standard_sandbox TO ROLE dev_role;

USE ROLE dev_role;
CREATE TABLE hr_prod_db.standard_sandbox.secret_table (data STRING);
GRANT SELECT ON TABLE hr_prod_db.standard_sandbox.secret_table TO ROLE PUBLIC;
--> Success! Developers can leak tables to the public.

-- 3. Managed Access Schema: Centralized control
USE ROLE SYSADMIN;
CREATE SCHEMA hr_prod_db.managed_sandbox WITH MANAGED ACCESS;
GRANT USAGE, CREATE TABLE ON SCHEMA hr_prod_db.managed_sandbox TO ROLE dev_role;

USE ROLE dev_role;
CREATE TABLE hr_prod_db.managed_sandbox.secured_table (data STRING);
GRANT SELECT ON TABLE hr_prod_db.managed_sandbox.secured_table TO ROLE PUBLIC;
--> ERROR: 'SQL execution error: Not authorized in a managed access schema.'
```

4. Hands-on Lab Exercises (Labs 3 & 4)

Lab 3: Cross-Database Joins (Secondary Session Roles)

Goal: Execute queries across two isolated databases without needing to combine privileges into one massive custom role or modify the structural RBAC tree.

```
-- 1. Set up isolated environments
USE ROLE SYSADMIN;
CREATE DATABASE IF NOT EXISTS sales_db;
CREATE SCHEMA IF NOT EXISTS sales_db.public_data;
CREATE TABLE IF NOT EXISTS sales_db.public_data.orders (id INT, customer STRING);
INSERT INTO sales_db.public_data.orders VALUES (1, 'Acme Corp');

CREATE DATABASE IF NOT EXISTS logistics_db;
CREATE SCHEMA IF NOT EXISTS logistics_db.public_data;
CREATE TABLE IF NOT EXISTS logistics_db.public_data.shipping (id INT, status STRING);
INSERT INTO logistics_db.public_data.shipping VALUES (1, 'SHIPPED');

-- 2. Build isolated roles
USE ROLE SECURITYADMIN;
CREATE ROLE sales_reader; CREATE ROLE logistics_reader;
GRANT ROLE sales_reader, logistics_reader TO ROLE sysadmin;
GRANT ROLE sales_reader, logistics_reader TO USER CURRENT_USER();

GRANT USAGE ON DATABASE sales_db TO ROLE sales_reader;
GRANT USAGE ON SCHEMA sales_db.public_data TO ROLE sales_reader;
GRANT SELECT ON TABLE sales_db.public_data.orders TO ROLE sales_reader;

GRANT USAGE ON DATABASE logistics_db TO ROLE logistics_reader;
GRANT USAGE ON SCHEMA logistics_db.public_data TO ROLE logistics_reader;
GRANT SELECT ON TABLE logistics_db.public_data.shipping TO ROLE logistics_reader;

-- 3. Query as sales_reader with secondary roles disabled
USE ROLE sales_reader;
USE SECONDARY ROLES NONE;
SELECT * FROM sales_db.public_data.orders o JOIN logistics_db.public_data.shipping s ON o.id =
s.id;
--> ERROR: 'Database LOGISTICS_DB does not exist or not authorized.'

-- 4. Enable secondary roles. Query now succeeds seamlessly!
USE SECONDARY ROLES ALL;
```

```
SELECT * FROM sales_db.public_data.orders o JOIN logistics_db.public_data.shipping s ON o.id = s.id;
```

Lab 4: Automated Governance (Future Grants)

Goal: Configure automated privilege assignment, verifying that roles receive permissions on tables that do not yet exist inside a schema.

```
USE ROLE SECURITYADMIN;
CREATE ROLE dev_auditor;
GRANT ROLE dev_auditor TO USER CURRENT_USER();
GRANT ROLE dev_auditor TO ROLE sysadmin;

-- Grant USAGE on database and schema
GRANT USAGE ON DATABASE hr_prod_db TO ROLE dev_auditor;
GRANT USAGE ON SCHEMA hr_prod_db.payroll TO ROLE dev_auditor;

-- Define FUTURE GRANT inside schema payroll
GRANT SELECT ON FUTURE TABLES IN SCHEMA hr_prod_db.payroll TO ROLE dev_auditor;

-- Switch to developer role to create a brand-new table
USE ROLE dev_role;
CREATE TABLE hr_prod_db.payroll.q3_audit (id INT, report STRING);
INSERT INTO hr_prod_db.payroll.q3_audit VALUES (101, 'Passed Security Validation');

-- Switch to auditor role and verify access is inherited automatically
USE ROLE dev_auditor;
SELECT * FROM hr_prod_db.payroll.q3_audit;
```


5. Authentication, Network Security & Exfiltration Control

To complete Domain 1, administrators must secure connections entering Snowflake and prevent unauthorized exports of sensitive data leaving Snowflake.

Authentication & User Administration (Domain 1.5)

Snowflake supports custom security integrations to manage external client configurations. Key admin tasks involve separating standard interactive accounts from machine-to-machine connections using **Key-Pair Authentication**.

- **User Classification:** Manage accounts by designating distinct user types: **PERSON** (standard logging users), **SERVICE** (non-interactive pipelines), and **NULL** (legacy/unspecified). Set RSA public keys directly onto SERVICE users.
- **SAML2 Integrations:** Establishes Single Sign-On (SSO) federation with corporate identity providers. These integrations can be configured to dynamically sync users and roles using SCIM (System for Cross-domain Identity Management).

Network Rules & Private Connections (Domain 1.6)

Network security uses **Network Rules** (reusable IP or domain lists) that are bundled inside **Network Policies**. When an administrator applies a network policy, Snowflake evaluates access top-down:

1. If a **User-Level** policy exists, it completely overrides the Account-Level policy for that user.
2. Administrators must keep port 443 open for standard query encryption, and port 80 open to transmit ****OCSP certification checks****.

Data Exfiltration Prevention (Domain 1.7)

To lock down data lakes and prevent malicious staging of data, Snowflake administrators must configure two critical security parameters:

Parameter Keyword	Operational Constraint & Security Posture
PREVENT_UNLOAD_TO_INLINE_URL	When set to TRUE, this parameter completely blocks users from executing COPY INTO commands to arbitrary, inline, or unapproved external cloud storage URLs, stopping egress leaks.
REQUIRE_STORAGE_INTEGRATION_FOR_STAGE_CREATION	When set to TRUE, this forces developers to use pre-authorized, secure Storage Integrations rather than passing raw access keys or cloud credentials during stage definition.

Auditing Active Network Parameters

Run these account-level commands to audit exfiltration risk profiles:

```
SHOW PARAMETERS LIKE 'PREVENT_UNLOAD_TO_INLINE_URL' IN ACCOUNT;  
SHOW PARAMETERS LIKE 'REQUIRE_STORAGE_INTEGRATION_FOR_STAGE_CREATION' IN ACCOUNT;
```

6. Exam-Realistic Diagnostic Scenario Drill

Use these scenarios to test your synthesis of Phase 1 mechanics. Walk through the questions, formulate your design choices, and study the detailed engineering rationale provided.

Scenario 1: The Secure Sandbox Leak

The Business Requirement: A security officer notices developers are creating test tables and frequently sharing them with the PUBLIC role to bypass standard ticket queues. You are ordered to instantly block developers from sharing *any* new table they create in the DEV_SANDBOX schema, while still allowing them to create tables freely.

The Engineering Solution: Convert the target schema to a **Managed Access Schema**. In standard discretionary schemas, object owners (the creator of a table) hold full grant authority. Converting to a Managed Access Schema strips object owners of grant privileges, restricting all grant actions to the schema owner (usually SYSADMIN) or roles with global MANAGE GRANTS.

```
-- Execute conversion to managed access
USE ROLE SYSADMIN;
ALTER SCHEMA dev_db.dev_sandbox ENABLE MANAGED ACCESS;
```

Scenario 2: The Multi-Region Storage Integrations Clone

The Business Requirement: Your team clones the raw staging schema `STAGE\_PROD` to a sandbox schema `STAGE\_SANDBOX` for analytics testing. After cloning, a developer active under role `SANDBOX\_DEV` runs a query against table `STAGE\_SANDBOX.AWS\_RAW` but receives an authorization error, despite having direct USAGE permissions granted on the sandbox schema container.

The Engineering Solution: In Snowflake's replication model, cloned containers **do not inherit the privileges assigned to the parent container itself**. The developer was granted USAGE on the parent schema `STAGE\_PROD`, but this permission did not transfer to `STAGE\_SANDBOX`. The administrator must manually grant USAGE on the newly cloned schema container itself, even though all nested child table privileges successfully cloned in place.

```
-- Grants must be reapplied at the container level
USE ROLE SECURITYADMIN;
GRANT USAGE ON SCHEMA dev_db.stage_sandbox TO ROLE sandbox_dev;
```

Scenario 3: SCIM and OAuth Configuration

For SCIM synchronization, remember that user profile mapping is completely automated. However, if a private connection (PrivateLink) is configured, standard OAuth client tokens must bypass public network policies. Ensure your SAML integrations allow explicit whitelisting of private connection VPC endpoints to maintain consistent user authorization flows.